Table des matières

Aide-mémoire du métier — Analytics Engineering 1
Le métier 1
Les couches 2
Qui lit qui 2
Concevoir avant de bâtir 2
Les dix cours 2
Le projet dbt 5
Deux environnements, dev et prod 6
Les commandes 6
Les pièges qui reviennent 7
Le vocabulaire 8
À retenir 9

Aide-mémoire du métier — Analytics Engineering

Formation Analytics Engineering — valable pour tous les projets

Une fiche à garder ouverte à côté des cours, dans n'importe quel projet de la formation : le métier, les couches, les dix cours, le projet dbt, les commandes, les pièges qui reviennent et le vocabulaire. Le cours NN porte le même nom dans chaque projet : seule la taille des données change.

Le métier

- Data Engineering charge les données brutes dans l'entrepôt : c'est la `RAW`.
- Vous, l'ingénieur analytique, en tirez des étoiles fiables, testées et documentées.
- Business Intelligence en tire les rapports.
- Chaque projet part d'un mandat : ce que l'entreprise vous demande, soit les questions précises auxquelles vos étoiles devront répondre, comme « combien se vend chaque produit, mois par mois ? ». Le mandat change d'un projet à l'autre.
- C'est l'ELT : charger d'abord, transformer ensuite, dans l'entrepôt. Data Engineering charge, vous transformez, avec dbt.

© 2026 iAlexMG. Tous droits réservés.

Les couches

- `RAW` : les données reçues, telles quelles, souvent tout en texte. On n'y touche jamais : les couches suivantes en font leurs propres versions.
- `STAGING` (`stg_`) : une vue par table reçue, nettoyée : renommée, typée, sans doublon, codes décodés, rien de personnel, et l'état courant tiré. Aucune jointure.
- `INTERMEDIATE` (`int_`) : les jointures et les règles métier, écrites une seule fois, comme ce qui compte pour une vente. Une vue, ou un `ephemeral`, que dbt glisse dans le modèle qui le lit sans créer d'objet.
- `MARTS` (`dim_`, `fct_`) : l'étoile que lisent les rapports, en tables. Une dimension dit par quoi on regarde ; un fait dit ce qu'on mesure.
- Au besoin : la `PRESENTATION` (`rpt_`), un résumé taillé pour un rapport, sans règle métier neuve ; un snapshot (`snp_`), l'histoire de ce qui change.
- Le préfixe dit la couche, jamais la forme : une vue et une table s'appellent `stg_…` de la même façon. Selon le projet, toutes les couches partagent un schéma, ou chacune a le sien (`raw`, `staging`, `marts`).

Qui lit qui

- `stg_` lit la `RAW`, par `source()`.
- `int_` lit des `stg_`, par `ref()`.
- `dim_` et `fct_` lisent des `stg_` ou des `int_`, jamais la `RAW`.

Concevoir avant de bâtir

Le Cours 02 dessine les `MARTS` avant que les Cours 03 et 04 bâtissent le `STAGING` et l'`INTERMEDIATE`. Concevoir et bâtir vont en sens inverse :

- Concevoir part de la fin. Le mandat dit quelle étoile il faut : le fait, son grain, ses dimensions, ses mesures. Ce plan dit ensuite quelles tables de la `RAW` serviront, ce que le `STAGING` devra nettoyer et quelles règles l'`INTERMEDIATE` devra poser.
- Bâtir part du début. Le Cours 03 monte le `STAGING` sur la `RAW` ; le Cours 04 pose l'`INTERMEDIATE`, puis bâtit l'étoile dessinée au Cours 02. Chaque couche lit celle d'en dessous.
- Le Cours 02 ne crée rien dans l'entrepôt : il livre un plan, et au besoin un essai en SQL qui disparaît ensuite. À sa fin, les `MARTS` n'existent pas encore.
- C'est l'ordre d'une maison : l'architecte dessine les pièces dont les occupants ont besoin, puis l'entrepreneur commence par la fondation.

Les dix cours

© 2026 iAlexMG. Tous droits réservés.

Chaque cours se range dans la couche dont il s'occupe, comme sur la page de la formation. Ce qui traverse les trois couches se range dans une bande : « Fiabiliser et faire tourner », puis « De bout en bout ». Rangés ainsi, les cours ne suivent pas leur numéro : le Cours 02 est sous les `MARTS`, qu'il dessine, même s'il vient avant les Cours 03 et 04.

`RAW` — reçue de Data Engineering

- Cours 00 — Générer les données sources
    - À faire : rien à écrire ; un script remplit la `RAW` à votre place, comme Data Engineering le fait en entreprise.
    - Deux chemins, la même `RAW` à l'arrivée :
        - vous avez fait Data Engineering : le script part de votre propre travail ;
        - sinon : le script génère lui-même les données sources, puis les charge.
    - Sortie : la `RAW` remplie, et son bilan « Conforme ».

`STAGING` — reprendre la `RAW`, une vue par table

- Cours 01 — Analyser la `RAW` reçue
    - À faire : lire avant de bâtir.
    - Entrée : la `RAW` et son dictionnaire.
    - Sortie : les tables utiles, et les défauts que le `STAGING` devra régler.
- Cours 03 — Monter le projet dbt et le `STAGING`
    - À faire : nettoyer chaque table sans rien juger.
    - Entrée : les tables de la `RAW`.
    - Sortie : le projet dbt, ses sources déclarées, et une vue `stg_` par table.

`INTERMEDIATE` — joindre, clarifier, préparer l'étoile

- Cours 04 — Bâtir l'étoile des ventes
    - À faire : poser la règle métier, bâtir l'étoile dessinée au Cours 02, la réconcilier avec le système d'origine.
    - Entrée : les vues `stg_`, et le plan du Cours 02.
    - Sortie : une `int_` ; puis, en fin de cours, les `dim_` (dont `dim_date`) et le `fct_`, qui montent déjà aux `MARTS`.

© 2026 iAlexMG. Tous droits réservés.

`MARTS` — concevoir et livrer les étoiles

- Cours 02 — Concevoir le modèle dimensionnel
  - À faire : dessiner l'étoile avant de coder. Il vient avant les Cours 03 et 04 : la section « Concevoir avant de bâtir » dit pourquoi.
  - Entrée : le mandat du projet.
  - Sortie : le plan de l'étoile : le fait, son grain (ce que représente une ligne), ses dimensions, ses mesures. Rien n'est encore créé dans l'entrepôt.
- Cours 06 — Modéliser les autres faits
  - À faire : choisir la bonne famille de fait.
  - Entrée : les autres journaux de la `RAW`.
  - Sortie : des faits d'un autre genre : d'accumulation, instantanés, sans mesure.

Fiabiliser et faire tourner — ce qui traverse les trois couches

- Cours 05 — Tester, historiser et documenter
  - À faire : rendre les contrôles permanents.
  - Entrée : l'étoile.
  - Sortie : des tests, un snapshot, la documentation.
- Cours 07 — Charger en incrémental
  - À faire : ne plus tout recalculer chaque nuit.
  - Entrée : une `RAW` qui arrive jour après jour.
  - Sortie : un fait qui n'ajoute que le nouveau.
- Cours 08 — Mettre en place l'intégration continue
  - À faire : faire tourner les vérifications seules.
  - Entrée : le dépôt.
  - Sortie : un linteur, et une CI (intégration continue) qui teste chaque changement et refuse ce qui casse.

De bout en bout — le projet dbt complet, assemblé

© 2026 iAlexMG. Tous droits réservés.

- Cours 09 — Rebâtir le projet de bout en bout
    - À faire : prouver que tout se refait.
    - Entrée : le dépôt, et la `RAW` seule.
    - Sortie : le projet rebâti, identique, documenté, étiqueté `v1.0`.

Le projet dbt

- Le code vit dans votre dépôt Git, `02_ae`, dans le dossier du projet. L'entrepôt, un fichier DuckDB, est à côté, hors du dépôt.
- `dbt_project.yml`, à la racine du dépôt : fait exister le projet et porte ses réglages (Cours 03).
- `profiles.yml`, hors du dépôt : dit à dbt où est l'entrepôt, et décrit ses cibles (Cours 03).
- `models/`, un sous-dossier par couche : un modèle est un fichier `.sql` qui contient un `select`, que dbt enveloppe en vue ou en table.
- `_sources.yml`, dans `models/staging/` : déclare les tables de la `RAW`, que le projet reçoit et ne modifie pas (Cours 03).
- Les autres `.yml` de `models/` (`_staging.yml`, `_marts.yml`…) : les descriptions et les tests.
- `snapshots/` : l'histoire d'une table qui change, version par version (Cours 05).
- `tests/` : les tests singuliers, des requêtes qui ne doivent rendre aucune ligne (Cours 05).
- `macros/` : du Jinja réutilisable, comme la garde du Cours 09.
- `.sqlfluff` et `ci/` : le linteur et la CI (Cours 08).
- `reconstruire.cmd` et `outils/` : refaire le projet de zéro, puis le comparer à l'original (Cours 09).
- Au besoin : `packages.yml`, des paquets comme `dbt_utils`, installés par `dbt deps` ; `seeds/`, un petit CSV que le projet écrit lui-même ; `_exposures.yml`, ce que Business Intelligence lit ; `LIVRAISON.md`, ce que le projet livre.

Pour appeler une table dans un modèle :

- une table de la `RAW` : `source('<source>', '<table>')`, déclarée dans `_sources.yml` ;
- un modèle ou un seed : `ref('<modèle>')`.

Ce sont ces appels qui donnent à dbt l'ordre de construction.

Les réglages de `dbt_project.yml` :

© 2026 iAlexMG. Tous droits réservés.

- Un réglage s'écrit avec un `+` devant (`+materialized: view`), pour le distinguer d'un nom de dossier.
- Le réglage le plus précis gagne : `dbt_project.yml`, puis le fichier `.yml` qui décrit le modèle, puis le `config()` du modèle lui-même.

Deux environnements, `dev` et `prod`

Un projet qui alimente de vrais rapports garde deux copies de ses couches, bâties par le même code. Le Cours 07 les met en place, dans un projet qui range chaque couche dans son schéma ; un petit projet, tout dans un schéma, n'a qu'un environnement.

- `dev`, votre atelier : vous y essayez vos changements sans déranger personne. Ses schémas portent le préfixe `dev_` : `dev_staging`, `dev_marts`.
- `prod`, la version officielle : c'est elle que lisent les rapports. Ses schémas portent leur nom seul : `staging`, `marts`.
- La cible se choisit à la commande. `dbt build` bâtit dans `dev`, la cible par défaut ; `dbt build --target prod` bâtit dans `prod`. Les deux cibles sont décrites dans `profiles.yml`, et la macro `generate_schema_name` ajoute le préfixe.
- Une exception : le snapshot écrit toujours dans le même schéma, `snapshots`, fixé dans sa configuration. Les deux cibles partagent donc la même histoire.
- Le piège : tout passe au vert, et les rapports n'ont pas bougé. Le `--target prod` a été oublié, et le travail est resté dans `dev_*`.

Les commandes

Dans l'Anaconda Prompt, environnement `formations` actif, ouvert dans le dossier du projet, après `set DBT_PROJECT_DIR=02_ae`.

- `dbt debug` : vérifie le profil, le projet et la connexion.
- `dbt run` : construit les modèles, sans les tests.
- `dbt test` : lance les tests ; `--select test_type:singular` pour les seuls tests singuliers.
- `dbt build` : construit et teste tout, dans l'ordre du graphe.
- `dbt compile` : montre le SQL que dbt exécuterait, Jinja résolue.
- `dbt ls --select <modèle>+` : ce qui dépend d'un modèle ; `+<modèle>`, ce dont il dépend.
- `dbt snapshot` : ajoute à l'histoire les versions qui ont changé.
- `dbt docs generate` : écrit la documentation et `target/manifest.json`.
- `dbt source freshness` : l'âge du dernier chargement de chaque source.

© 2026 iAlexMG. Tous droits réservés.

- `--full-refresh` : reconstruit un modèle incrémental en entier.
- `dbt run-operation exiger_entrepot_neuf` : la garde, qui refuse un entrepôt qui porte déjà le projet.
- `sqlfluff lint 02_ae\models 02_ae\tests 02_ae\snapshots --config 02_ae\.sqlfluff` : vérifie la forme du SQL ; `fix` à la place de `lint` corrige ce qui se corrige.
- `02_ae\reconstruire.cmd` : refait le projet de zéro, sur un entrepôt qui n'a que la `RAW`.
- `git -C 02_ae …` : Git sur le dépôt, sans changer de dossier ; `git tag -a v1.0` pose l'étiquette d'un état livré.
- `duckdb -ui` : le notebook, pour lire l'entrepôt, attaché en lecture seule.
- Au besoin : `dbt deps` installe les paquets ; `dbt seed` charge les seeds ; `--target prod` bâtit la version officielle ; `dbt run-operation nettoyer_dev` jette les schémas `dev_*`.

Les pièges qui reviennent

- `No dbt_project.yml found at expected path` : la fenêtre a oublié `set DBT_PROJECT_DIR=02_ae`.
- dbt ne peut pas ouvrir l'entrepôt, « utilisé par un autre processus » : le notebook ou la ligne de commande DuckDB le tient. Un seul processus écrit à la fois : détacher l'entrepôt du notebook (`detach`) ou quitter la ligne de commande (`.exit`), puis relancer dbt.
- `cannot access local variable 'summary_message'` : `profiles.yml` existe, mais il est vide.
- Les vues atterrissent dans `staging_staging` : le projet range ses couches dans des schémas, et la macro `generate_schema_name` manque.
- Une vue qui passe, puis casse au `select *` : DuckDB ne calcule que les colonnes qu'on lit, et l'erreur d'une autre colonne attend son tour. Vérifier les colonnes et leurs types par `describe`.
- Le chiffre d'affaires sort trop haut : la `RAW` garde toutes ses livraisons, et l'état courant n'a pas été tiré ; ce qui est livré deux fois compte deux fois.
- Les mois ou les jours sortent en anglais : DuckDB les nomme en anglais ; le décodage français se fait une seule fois, dans `dim_date`.
- Un jour de trop ou de moins sur un horodatage : DuckDB tire le jour dans le fuseau du poste ; la conversion s'écrit en toutes lettres (`at time zone`).
- Un `accepted_values` au vert sur une colonne vide : il ignore les valeurs absentes ; un `not_null` va toujours à côté.
- Des versions qui finissent avant de commencer : les lots du snapshot ont été rejoués dans le désordre, ou par-dessus une histoire.
- Des lignes perdues sans un message, en incrémental : le filtre porte sur la date de l'événement au lieu de la date d'arrivée.
- Tout est au vert et les rapports n'ont pas bougé : `--target prod` oublié (section « Deux environnements, `dev` et `prod` »).

© 2026 iAlexMG. Tous droits réservés.

- `Error: No such option '--profiles-dir'.` au linteur : le profil du linteur se déclare dans `.sqlfluff`, sous `[sqlfluff:templater:dbt]`.
- Une comparaison de vues qui rend toujours 0 : une vue DuckDB nomme sa base ; attachée à côté d'un entrepôt du même nom, elle lit les tables de l'autre.

Le vocabulaire

- Entrepôt de données : une base pensée pour analyser, et non pour faire tourner une application ; ici, un fichier DuckDB.
- OLTP et OLAP : le traitement transactionnel en ligne, celui du système qui enregistre une vente, contre le traitement analytique en ligne, celui qui les compte par millions ; la formation fait le second.
- Mandat : ce que l'entreprise demande au projet, les questions auxquelles les étoiles doivent répondre ; il change d'un projet à l'autre.
- dbt : un logiciel en ligne de commande qui compile du SQL et du Jinja, l'exécute dans l'ordre et le teste ; le « T » de l'ELT.
- Modèle : un fichier `.sql` qui contient un `select`, et que dbt matérialise.
- Matérialisation : la forme que prend un modèle : `view`, `table`, `ephemeral`, `incremental`.
- Source : une table que le projet reçoit et ne fabrique pas, appelée par `source()`.
- `ref()` : l'appel d'un modèle ou d'un seed ; c'est lui qui trace le graphe.
- DAG : le graphe orienté acyclique des dépendances, que dbt tire des `ref()` et des `source()` et qu'il parcourt dans l'ordre.
- Cible : un environnement décrit dans `profiles.yml` et choisi par `--target`, comme `dev`, `prod` ou `ci`.
- État courant : la dernière version de chaque clé, tirée d'une `RAW` qui garde toutes ses livraisons.
- Grain : ce que représente une ligne d'un fait, déclaré avant les colonnes.
- Fait, dimension, étoile : les événements chiffrés au centre (`fct_`, de l'anglais *fact*), leur contexte autour (`dim_`), reliés par des clés.
- Familles de faits : de transaction, un événement ; d'accumulation, un dossier qui franchit des jalons ; instantané périodique, un état à intervalle régulier ; sans mesure, un événement qui n'a que ses clés.
- Clé de substitution : un identifiant fabriqué par l'entrepôt, souvent par hachage, que le fait porte à la place de la clé de la source.
- Dimension conforme : une dimension partagée par plusieurs faits, définie une fois ; c'est elle qui permet de croiser deux faits.
- Dimension dégénérée : une clé que le fait porte sans dimension derrière elle, comme un numéro de commande.

© 2026 iAlexMG. Tous droits réservés.

- Matrice de bus : le tableau des processus contre les dimensions conformes qu'ils partagent.
- Type 1, type 2 : écraser l'ancienne valeur d'un attribut, ou garder une version datée par valeur.
- Snapshot : le mécanisme de dbt qui garde les versions d'une table qui change.
- Seed : un petit fichier CSV versionné dans le dépôt, chargé par `dbt seed`.
- Test générique, singulier, unitaire : un test déclaré en YAML sur une colonne ; une requête qui ne doit rien rendre ; un modèle essayé sur des entrées choisies.
- Témoin indépendant : un test qui compare l'étoile à un chiffre tiré d'ailleurs, sans passer par elle, comme le système d'origine ou un compteur.
- Contrat : les colonnes et les types qu'un modèle promet, vérifiés avant de créer la table.
- `exposure` : la déclaration, dans le projet, de ce qu'un consommateur lit.
- Incrémental : un modèle qui n'ajoute que ce qui est arrivé depuis la dernière exécution, sur une clé unique.
- Fenêtre de reprise : les jours qu'un modèle incrémental relit à chaque passe, pour rattraper les retards.
- Fraîcheur : l'âge du dernier chargement d'une source, comparé à sa cadence.
- `manifest.json` : le catalogue que dbt écrit (modèles, colonnes, descriptions, tests, graphe) ; ce qu'un outil ou un agent lit.
- Linteur : l'outil qui vérifie la forme du code, ici sqlfluff.
- Reconstruction : le projet refait de zéro depuis son dépôt, sur un entrepôt qui n'a que la `RAW`.
- Garde : une vérification qui refuse de lancer une opération au mauvais endroit.

À retenir

- La `RAW` ne change jamais.
- Le `STAGING` garde une ligne par objet reçu, juste propre.
- Une jointure ou une règle métier, c'est l'`INTERMEDIATE`.
- Les `MARTS` ne lisent jamais la `RAW`.
- On conçoit en partant des `MARTS`, on bâtit en partant de la `RAW`.
- Un fait se vérifie contre un chiffre tiré d'ailleurs, le témoin, avant de se livrer.

© 2026 iAlexMG. Tous droits réservés.